



Введение

В продакшене у вас не будет доступа к консоли IDE. У вас будет 50 контейнеров, работающих на разных серверах.

- Как понять, почему запрос тормозит?
- Где произошла ошибка, если логов гигабайты?
- Хватает ли памяти приложению?

Распределенная трассировка

Мы писали свой фильтр для CorrelationId. Это полезно для обучения, но в индустрии используют стандарты. С выходом Spring Boot 3 стандартом стал Micrometer Tracing. Он автоматически:

1. Генерирует TracelId (общий ID для всей цепочки) и SpanId (ID конкретной операции).
2. Прокидывает их через HTTP заголовки.
3. Прокидывает их через заголовки сообщений RabbitMQ и метаданные gRPC.
4. Отправляет данные в систему визуализации (Zipkin, Jaeger).

Метрики

Spring Boot Actuator предоставляет эндпоинт /actuator/prometheus, который отдает метрики в формате, понятном системе мониторинга Prometheus.

Практическая часть

Нам предстоит модифицировать все наши сервисы.

Вам нужно добавить одни и те же зависимости в pom.xml для каждого из 4-х сервисов (demo-rest, analytics-service, notification-service, audit-service)

<dependency>

<groupId>org.springframework.boot**</groupId>**

<artifactId>spring-boot-starter-

actuator**</artifactId>**

</dependency>



| Docker, logging, tracing

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-
prometheus</artifactId>
</dependency>
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-tracing-bridge-
brave</artifactId>
</dependency>
<dependency>
    <groupId>io.zipkin.reporter2</groupId>
    <artifactId>zipkin-reporter-brave</artifactId>
</dependency>
<dependency>
    <groupId>net.logstash.logback</groupId>
    <artifactId>logstash-logback-
encoder</artifactId>
    <version>7.4</version>
</dependency>
```

Создайте файл `src/main/resources/logback-spring.xml` в каждом сервисе. Этот файл настроит Logback писать JSON.

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
```



```
<include
resource="org/springframework/boot/logging/logback/defaults.
xml"/>
<appender name="CONSOLE"
class="ch.qos.logback.core.ConsoleAppender">
    <encoder
class="net.logstash.logback.encoder.LogstashEncoder">
        <includeMdcKeyName>traceId</includeMdcKeyName>
        <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
</appender>
<root level="INFO">
    <appender-ref ref="CONSOLE"/>
</root>
</configuration>
```

Теперь логи в консоли будут выглядеть как {"@timestamp":"...", "message":"...", "traceId":"..."}

В каждом сервисе в application.properties (или yml) нужно добавить настройки для Actuator и Zipkin.

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

management.tracing.sampling.probability=1.0
management.zipkin.tracing.endpoint=http://zipkin:9411
/api/v2/spans
```



`management.metrics.export.prometheus.enabled=true`

В проекте demo-rest удалите или отключите наш самописный LoggingAndTracingFilter (к примеру я закомментил аннотацию @Component) а в RabbitMQConfig уберите фабрику и оставьте только конвертеры, так как Spring Boot + Micrometer теперь сами будут управлять заголовками.

В корне каждого проекта создайте файл с именем Dockerfile (без расширения). Содержимое будет одинаковым практически, за исключением порта (так как мы используем стандартную сборку).

Вот пример для demo rest, остальное в сборке проекта смотрите

```
# Используем легковесный образ JRE 21 на базе Alpine Linux
FROM eclipse-temurin:21-jre-alpine
```

```
# Создаем рабочую директорию
WORKDIR /app
```

```
# Создаем пользователя 'javauser', чтобы не запускать
приложение от root (Security Best Practice)
RUN addgroup -S javauser && adduser -S javauser -G javauser
```

```
# Аргумент для поиска JAR файла.
```

```
# Мы ищем любой JAR, но исключаем *-sources.jar или *-
javadoc.jar если они есть.
```

```
# Перед сборкой еще сделайте mvn clean, в папке target будет
лежать нужный нам fat-jar.
```

```
ARG JAR_FILE=target/*.jar
```



| Docker, logging, tracing

Копируем JAR внутрь образа под именем app.jar

COPY \${JAR_FILE} app.jar

Меняем владельца файла

RUN chown javauser:javauser /app/app.jar

Переключаемся на пользователя

USER javauser

Документируем порт (этот порт должен совпадать с server.port в properties)

EXPOSE 8080

Запускаем приложение

Использование ехес формы ["..."] позволяет корректно передавать сигналы остановки (SIGTERM)

ENTRYPOINT ["java", "-jar", "app.jar"]

Важно перейти от физических сборок в lib к сборке в рамках репозитория. У нас это контракты – book api и event'ы. Поэтому импорты заменяем на корректные:

<dependency>

<groupId>edu.rutmiit.demo**</groupId>**

<artifactId>events-contract**</artifactId>**

<version>1.0-SNAPSHOT**</version>**

</dependency>



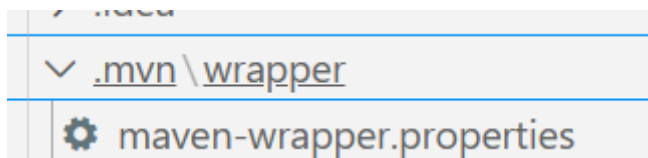
```
<dependency>
  <groupId>edu.rutmiit.demo</groupId>
  <artifactId>books-api-
contract</artifactId>
  <version>1.0</version>
</dependency>
<dependency>
```

А в проекте с контрактами нужно собрать в локальные репозитории,
mvn install

или для Windows:

./mvnw.exe install

Обратите внимание что у вас должны быть в проекте



Скопируйте их просто в себе в проект с моего (последнее, это если вы на Windows)

Перед запуском Docker Compose вам нужно собрать все проекты в .jar файлы локально. Выполните mvn clean package -DskipTests в каждой папке проекта (или в windows если у вас есть врапперы)



./mvnw.exe clean package -DskipTests



| Docker, logging, tracing

Docker

В папке со всеми проектами создайте docker-compose.yml и prometheus.yml:

docker-compose.yml

```
version: "3.8"
```

```
services:
```

```
# Инфраструктура: Брокер сообщений
```

```
rabbitmq:
```

```
  image: rabbitmq:3.13-management
```

```
  ports:
```

- "5672:5672"
- "15672:15672"

```
  networks:
```

- microservices-net

```
  healthcheck:
```

```
    test: ["CMD", "rabbitmq-diagnostics", "-q",  
"ping"]
```

```
    interval: 10s
```

```
    timeout: 5s
```

```
    retries: 5
```

```
# Инфраструктура: Трассировка
```

```
zipkin:
```

```
  image: openzipkin/zipkin
```

```
  ports:
```



| Docker, logging, tracing

- "9411:9411"

networks:

- microservices-net

Инфраструктура: Метрики

prometheus:

image: prom/prometheus

volumes:

-

./prometheus.yml:/etc/prometheus/prometheus.yml

ports:

- "9090:9090"

networks:

- microservices-net

Микросервисы

analytics-service:

build: ./analytics-service

ports:

- "9091:9090" # gRPC порт

environment:

- SPRING_RABBITMQ_HOST=rabbitmq

depends_on:

rabbitmq:

condition: service_healthy

networks:



| Docker, logging, tracing

- `microservices-net`

`notification-service:`

`build: ./ws`

`ports:`

- `"8083:8083"`

`environment:`

- `SPRING_RABBITMQ_HOST=rabbitmq`
- `SERVER_PORT=8083`

`depends_on:`

`rabbitmq:`

`condition: service_healthy`

`networks:`

- `microservices-net`

`audit-service:`

`build: ./audit-service`

`environment:`

- `SPRING_RABBITMQ_HOST=rabbitmq`
- `SERVER_PORT=8082`

`depends_on:`

`rabbitmq:`

`condition: service_healthy`

`networks:`

- `microservices-net`



demo-rest:

build: ./demo-rest

ports:

- "8080:8080"

environment:

- SPRING_RABBITMQ_HOST=rabbitmq

Указываем адрес gRPC сервиса внутри сети

Docker

-

GRPC_CLIENT_ANALYTICS_SERVICE_ADDRESS=static://analytics-service:9090

depends_on:

rabbitmq:

condition: service_healthy

analytics-service:

condition: service_started

networks:

- microservices-net

networks:

microservices-net:

driver: bridge

prometheus.yml

global:



scrape_interval: 5s

scrape_configs:

- job_name: "demo_app"
metrics_path: "/actuator/prometheus"
static_configs:
 - targets:
 - [
"demo-rest:8080",
"notification-service:8083",
"audit-service:8082",
"analytics-service:8081",
]

Запуск

В корневой папке все композ файл.

docker-compose up --build -d

Логи:

docker-compose logs -f

Делаем POST запрос (к примеру через Postman)

POST <http://localhost:8080/api/users/999/rate>

Далее идем смотреть трейсы:

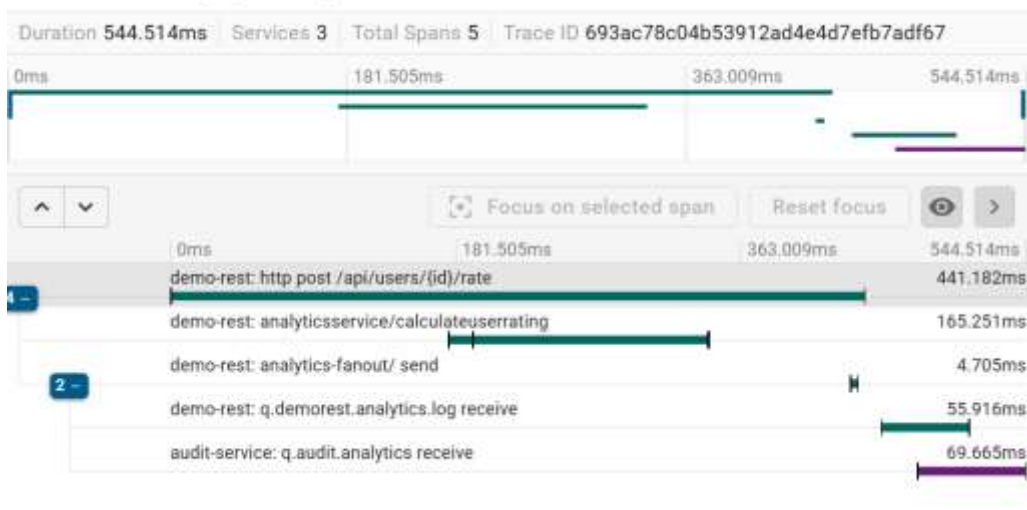
<http://localhost:9411>

Нажмите "Run Query":



Docker, logging, tracing

demo-rest: http post /api/users/{id}/rate



Проверка метрик:

Откройте <http://localhost:9090>.

Введите в поиск `http_server_requests_seconds_count` и нажмите Execute.

Вы увидите количество запросов к каждому сервису.

